

Python Programlama

Ders 10

Ali Mertcan KOSE Msc.

`amertcankose@ticaret.edu.tr`

İstanbul Ticaret Üniversitesi



İSTANBUL TİCARET
ÜNİVERSİTESİ

NumPy, bilimsel hesaplama ve veri analizi için kullanılan, “Numerical Python” (Sayısal Python) ifadesinin kısaltması olan bir Python modülüdür.

NumPy şunları sağlar:

- Hızlı ve verimli veri depolama ve işleme için çok boyutlu diziler (veri yapıları).
- Verileri manipüle etmek için kullanışlı fonksiyonlar.
- Programlara veri aktarma ve programlardan veri alma araçları.

NumPy'nin veri analizi için sunduğu işlevler şunlardır:

- Veri manipülasyonu için dizi (array) işlemleri.
- Sıralama, benzersiz (unique) değerler ve küme işlemleri gibi yaygın algoritmalar.
- Betimleyici istatistikler, veri toplama ve özetleme işlemleri.
- Veri setlerinin birleştirilmesi ve eklenmesi.
- Döngüler yerine koşullu mantığın dizi ifadeleriyle ifade edilmesi.

NumPy İşlevselliğini Kullanma

NumPy paketini kullanmak için, öncelikle daha önce olduğu gibi içe aktarılması (import edilmesi) gerekir.

`import numpy as np` -> Paket içindeki işlevselliği kullanmak istediğimizde, şu sözdizimini kullanacağız: `np.functionName(...)`

NumPy Veri Yapısı: ndarray

Bir veri yapısında birden fazla değeri bir arada depolar.

Bellekte sabit boyutlu bir dizi olarak bulunur ve aynı türden verileri içerir. Listelerden farklı olarak, tüm öğeler aynı türde olmalıdır, bu nedenle bir dizi farklı türleri içeremez.

Bir NumPy dizisinin boyutu oluşturulduğunda sabittir; bir diziyi büyütme veya küçültme mümkün değildir.

Eleman ekleme, ekleme veya silme işlemleri yapılamaz.

NumPy dizileri değiştirilebilir (mutable) olduğundan, dizide depolanan değerler değiştirilebilir.

Döngü kullanmadan, eleman bazında işlemlere olanak tanıyan yerleşik işlevselliklere sahiptir.

NumPy Dizi (Array) Özellikleri

Bir NumPy dizisi, aynı türdeki değerlerin bir satırı veya ızgarasıdır ve sıfırdan küçük olmayan tam sayılardan oluşan bir “tuple” ile indekslenir. Her dizinin aşağıdaki özellikleri vardır:

- `dtype` → Dizideki veri türü
- `ndim` → Boyut sayısı
- `shape` → Her bir boyutun boyutu
- `size` → Dizideki toplam eleman sayısı

ndarray oluşturmak için fonksiyonlar

- `array()`: Verilen değerler dizisiyle bir dizi (array) oluşturur.
- `arange()`: Belirtilen aralıktaki değerlerle bir dizi oluşturur.
- `linspace()`: Belirtilen aralıkta, n adet eşit aralıklı değerle bir dizi oluşturur.
- `empty()`: Belirtilen şekle sahip, rastgele (boş) öğelerle dolu bir dizi oluşturur.

ndarray oluşturmak için fonksiyonlar

- `zeros()`: Belirtilen şekle sahip, sıfırlarla dolu bir dizi oluşturur.
- `ones()`: Belirtilen şekle sahip, birlerle dolu bir dizi oluşturur.
- `rand()` `randint()`: Belirtilen şekle sahip, rastgele üretilmiş değerlerle bir dizi oluşturur.

NumPy Dizileri (Arrays) Oluşturma

NumPy dizilerini Python listelerinden başlatabilir ve elemanlara köşeli parantezler kullanarak erişebiliriz. Bir listeden dizi oluşturmak için `array()` komutunu kullanabilir ve listeyi bir parametre olarak geçirebiliriz.

Örnek Bir boyutlu bir dizi oluşturma:

```
import numpy as np  
  
z = np.array([4,7,6,9,2])  
z Out[13]: array([4, 7, 6, 9, 2])  
print(z)
```

Özniteliklerle İlgili Örnekler

```
r = np.array([87,33,5,16,9,17])
```

```
r
```

```
Out[29]: array([87, 33, 5, 16, 9, 17])
```

```
r.dtype
```

```
Out[30]: dtype('int32')
```

Özniteliklerle İlgili Örnekler

```
r.ndim
```

```
Out[31]: 1
```

```
r.shape
```

```
Out[32]: (6,)
```

```
r.size
```

```
Out[33]: 6
```

NumPy Dizileri Oluşturma

NumPy dizileri, `arange` fonksiyonu kullanılarak da oluşturulabilir. **`arange`** fonksiyonu, başlangıç, bitiş ve adım değerlerini alır ve bu değerleri içeren bir dizi oluşturur.

Örnek:

```
b = np.arange(1,10,2)
```

```
b
```

```
Out[67]: array([1, 3, 5, 7, 9])
```

NumPy Dizileri Oluşturma

```
t = np.arange(1,10)
```

```
t
```

```
Out[65]: array([1, 2, 3, 4, 5, 6, 7, 8, 9])
```

linspace()

Fonksiyon, eşit aralıklı ondalıklı (float) değerler oluşturur. Başlangıç, bitiş ve değer sayısını parametre olarak alır. Belirtilen aralıkta verilen sayıda değer üretir.

linspace()

```
In [31]: np.linspace?
         np.linspace(-50, 50, 11)

Out[31]: array([-50., -40., -30., -20., -10.,   0.,  10.,  20.,  30.,  40.,  50.])

In [33]: np.linspace(-10, 10)

Out[33]: array([-10.    , -9.5918, -9.1837, -8.7755, -8.3673, -7.9592,
        -7.551  , -7.1429, -6.7347, -6.3265, -5.9184, -5.5102,
        -5.102  , -4.6939, -4.2857, -3.8776, -3.4694, -3.0612,
        -2.6531, -2.2449, -1.8367, -1.4286, -1.0204, -0.6122,
        -0.2041,  0.2041,  0.6122,  1.0204,  1.4286,  1.8367,
         2.2449,  2.6531,  3.0612,  3.4694,  3.8776,  4.2857,
         4.6939,  5.102  ,  5.5102,  5.9184,  6.3265,  6.7347,
         7.1429,  7.551  ,  7.9592,  8.3673,  8.7755,  9.1837,
         9.5918, 10.    ])
```

Figure 1: linspace()

Özel Diziler Oluşturma

Başlangıç değerlerine sahip diziler oluşturmak için daha önce array ve arange fonksiyonlarını kullandık. Ayrıca, diğer varsayılan değerlerle (örneğin, ones, zeros) diziler oluşturmak için kullanabileceğimiz özel fonksiyonlar da mevcuttur.

Özel Diziler Oluşturma

Function	Output
array	Bir NumPy dizi (array) nesnesi oluşturur.
arange	Belirtilen aralıkta (başlangıç, bitiş, adım) değerlerle bir NumPy dizisi oluşturur.
ones	Belirtilen şekle sahip, tüm elemanları bir olan bir NumPy dizisi oluşturur.
zeros	Belirtilen şekle sahip, tüm elemanları sıfır olan bir NumPy dizisi oluşturur.

1D-Boyutlu Diziler Oluşturma: zeros() ve ones()

```
In [1]: import numpy as np

In [2]: np.zeros(10)
Out[2]: array([0., 0., 0., 0., 0., 0., 0., 0., 0., 0.])

In [4]: np.ones(10)
Out[4]: array([1., 1., 1., 1., 1., 1., 1., 1., 1., 1.])
```

Figure 2: zeros() ve ones()

Rastgele Diziler Oluşturma

NumPy'de, rastgele diziler oluşturmak için kullanılan `random` adlı bir alt modül bulunmaktadır.

Örnek:

`rand(n)`: 0 ile 1 arasında n adet rastgele değer içeren bir dizi oluşturur.

`randint()`: `low` (dahil) ile `high` (hariç) arasında n adet rastgele tamsayı değeri içeren bir dizi oluşturur.

Rastgele Diziler Oluşturma

```
import numpy as np  
r_vals = np.random.rand(10)  
print(r_vals)  
r_vals = np.random.randint(1,26,12)  
print(r_vals)
```

NumPy Dizilerine Erişim ve Güncelleme

Erişim elements:

`z[0] -> 4`

`z[3] -> 9`

Güncelleme elements:

`z[2] = 5`

`z`

Out[19]: `array([4, 7, 5, 9, 2])`

NumPy Dizilerini Dilimleme (Slicing)

Listelerde olduğu gibi, NumPy dizilerinin belirli bölümlerine başlangıç:bitis:adım sözdizimini kullanarak erişebiliriz.

Hatırlatma:

- bitis değeri belirtilen eleman dahil edilmez (hariç tutulur).
- Herhangi bir değer atlanırsa, varsayılan değerler şunlardır:
- başlangıç: 0
- bitis: dizinin sonu
 - adım: 1

NumPy Dizilerini Dilimleme (Slicing)

Örnekler:

z

Out[22]: array([4, 7, 5, 9, 2])

z[1:4:2]

Out[21]: array([7, 9])

z[1:4]

Out[23]: array([7, 5, 9])

z[1::2]

Out[24]: array([7, 9])

z[:4]

Out[25]: array([4, 7, 5, 9])

İki Boyutlu Diziler Oluşturma

İki boyutlu NumPy dizileri, dizilerin dizisi olarak düşünülebilir. Diziyi oluşturmak için `array` komutunu kullanıp bir listeyi parametre olarak geçirdik. Şimdi, bir liste yerine listelerin listesini parametre olarak geçirebiliriz.

Örnek:

```
m = np.array([[3,5,2,4],[7,6,8,8],[1,6,7,7]]) m Out[35]:  
array([[3, 5, 2, [7, 6, 8, 8], [1, 6, 7, 7]])
```

2D Boyutlu Diziler Oluşturma: zeros() ve ones()

```
In [1]: import numpy as np
```

```
In [3]: np.zeros((3,8))
```

```
Out[3]:
```

```
array([[0., 0., 0., 0., 0., 0., 0., 0.],  
       [0., 0., 0., 0., 0., 0., 0., 0.],  
       [0., 0., 0., 0., 0., 0., 0., 0.]])
```

```
In [5]: np.ones((3,8))
```

```
Out[5]:
```

```
array([[1., 1., 1., 1., 1., 1., 1., 1.],  
       [1., 1., 1., 1., 1., 1., 1., 1.],  
       [1., 1., 1., 1., 1., 1., 1., 1.]])
```

```
In [6]: np.ones((2,3,2))
```

```
Out[6]:
```

```
array([[[1., 1.],  
        [1., 1.],  
        [1., 1.]],  
       [[1., 1.],  
        [1., 1.],  
        [1., 1.]])
```

Figure 3: 2D-zeros() ve ones()

Elementlere Erişim: İki Boyutlu Diziler (2D Arrays)

Syntax: `arrName[ row, col]`

Örnekler:

`m[1,2]` Out[43]: 9

`m[2,1] = 12`

`m`

Out[45]:

`array([[ 3, 5, 2, 4], [ 7, 6, 9, 8], [ 1, 12, 0, 7]])`

Elemanlara Erişim Dilimleme (Slicing)

Ayrıca, başlangıç:bitis:adım sözdizimini her bir boyut (satır, sütun) için bir virgül ile ayırarak, iki boyutlu dizileri (2D diziler) dilimleyebiliriz.

Örnekler:

```
m[ 1:2,1:2]
```

```
Out[46]: array([[6]])
```

```
m[ 1, 1]
```

```
Out[48]: 7
```

Elemanlara Erişim Dilimleme (Slicing)

```
y = m[2, :]
```

```
y
```

```
Out[50]: array([ 1, 12, 0, 7])
```

```
t = m[2, ]
```

```
t Out[52]: array([ 1, 12, 0, 7])
```

```
w = m[2:0: 2,1:3]
```

```
w
```

```
Out[56]: array([[12, 0]])
```

```
20
```

Element Düzeyinde İşlemler

Listelerden farklı olarak, dizilerde döngü kullanmadan eleman düzeyinde işlemler yapmamıza olanak tanır. Dizilere aritmetik işlemler uyguladığımızda, bu işlem dizideki her bir elemana ayrı ayrı uygulanır.

Python Dizileri Matematiksel Fonksiyonlar

Temel matematiksel fonksiyonlar, diziler üzerinde eleman düzeyinde çalışır ve hem operatör aşırı yüklemeleri (operator overloads) hem de NumPy modülündeki fonksiyonlar olarak kullanılabilir.

Temel matematiksel fonksiyonlar, diziler üzerinde eleman bazında çalışır ve hem operatör aşırı yüklemeleri (operator overloads) hem de NumPy modülündeki fonksiyonlar olarak kullanılabilir.

Python Dizileri Matematiksel Fonksiyonlar

Operator	Equivalent ufunc	Description
+	<code>np.add</code>	Addition (e.g., $1 + 1 = 2$)
-	<code>np.subtract</code>	Subtraction (e.g., $3 - 2 = 1$)
-	<code>np.negative</code>	Unary negation (e.g., -2)
*	<code>np.multiply</code>	Multiplication (e.g., $2 * 3 = 6$)
/	<code>np.divide</code>	Division (e.g., $3 / 2 = 1.5$)
//	<code>np.floor_divide</code>	Floor division (e.g., $3 // 2 = 1$)
**	<code>np.power</code>	Exponentiation (e.g., $2 ** 3 = 8$)
%	<code>np.mod</code>	Modulus/remainder (e.g., $9 \% 4 = 1$)

Figure 4: `2D-zeros()` ve `ones()`

NumPy Dizileri Yerleşik Fonksiyonlar

NumPy dizilerinde, bir dizi üzerinde istatistiksel hesaplamalar yapmak için birçok yerleşik fonksiyon bulunmaktadır.

NumPy Dizileri Yerleşik Fonksiyonlar

Function	Tanım
<code>np.sum()</code>	Verilen dizideki değerlerin toplamını hesaplar.
<code>np.mean()</code>	Verilen dizideki değerlerin ortalamasını (mean) hesaplar.
<code>np.max()</code>	Verilen dizideki en büyük değeri hesaplar.
<code>np.min()</code>	Verilen dizideki en küçük değeri hesaplar.
<code>np.argmax()</code>	Verilen dizideki en büyük değer indeksini hesaplar.
<code>np.argmin()</code>	Verilen dizideki en küçük değer indeksini hesaplar.

NumPy Dizileri Yeniden Şekillendirme (Reshaping) ve Transpoz (Transpose)

Dizilerdeki bir diğer kullanışlı işlem, dizilerin yeniden şekillendirilmesidir (reshaping). Bu işlem, `reshape()` fonksiyonu kullanılarak gerçekleştirilir.

Örnek: `x = np.arange(1, 10)` dizisini 3x3 boyutunda bir diziye dönüştürme:

```
In [4]: M = x.reshape((3, 3))  
M
```

```
Out[4]: array([[1, 2, 3],  
               [4, 5, 6],  
               [7, 8, 9]])
```

Figure 5: Yeniden Şekillendirme (Reshaping)

NumPy Dizileri Yeniden Şekillendirme (Reshaping) ve Transpoz (Transpose)

Ayrıca, matrisin transpozunu bulma gibi yaygın matris işlemleri de yapabilirsiniz. Bir matrisin transpozu, satır ve sütun indislerini değiştirerek yeni bir matris oluşturur.

Örnek: `.T` kullanarak transpozu hesaplayın.

```
In [5]: M.T
```

```
Out[5]: array([[1, 4, 7],  
               [2, 5, 8],  
               [3, 6, 9]])
```

Figure 6: Transpoz (Transpose)

NumPy Dizileri Matris İşlemleri

NumPy, tipik matris işlemlerini verimli bir şekilde gerçekleştirebilir. Örnekler: `np.dot` kullanarak matris-vektör çarpımı.

```
In [6]: np.dot(M, [5, 6, 7])
```

```
Out[6]: array([ 38,  92, 146])
```

and even more sophisticated operations like eigenvalue decomposition:

```
In [7]: np.linalg.eigvals(M)
```

```
Out[7]: array([ 1.61168440e+01, -1.11684397e+00, -1.30367773e-15])
```

Figure 7: Matris

NumPy Dizileri Birleştirme (Concatenation)

Ayrıca, birden fazla diziyi tek bir dizide birleştirmek veya tersine, tek bir diziyi birden çok diziyeye bölmek de mümkündür. NumPy'de iki diziyi birleştirme işlemi, çoğunlukla aşağıdaki fonksiyonlarla gerçekleştirilir:

```
import numpy as np

x = np.array([1,2,3])
y = np.array([3,2,1])
print(np.concatenate([x,y]))
```

Figure 8: Birleştirme

NumPy Dizileri Birleştirme (Concatenation)

`np.concatenate()` İki veya daha fazla diziyi veya iki boyutlu dizileri birleştirmek için de kullanılabilir.

```
z = [99, 99, 99]
print(np.concatenate([x, y, z]))
```

Output:

```
[ 1  2  3  3  2  1 99 99 99]
```

```
grid = np.array([[1, 2, 3],[4, 5, 6]])
print(np.concatenate([grid, grid])) #default axis = 0
```

```
[[1 2 3]
 [4 5 6]
 [1 2 3]
 [4 5 6]]
```

```
grid = np.array([[1, 2, 3],[4, 5, 6]])
print(np.concatenate([grid, grid], axis = 1))
```

```
[[1 2 3 1 2 3]
 [4 5 6 4 5 6]]
```

Figure 9: Birleştirme

concatenate fonksiyonu, birleştirdiğiniz diziler aynı boyut sayısına (bir veya iki boyut) sahipse kullanılabilir. Eğer farklı boyutlara sahip dizileri birleştirmek istiyorsanız, vstack veya hstack fonksiyonlarını kullanmalısınız.

- vstack: İki veya daha fazla diziyi dikey olarak (üst üste) yığar ve yeni bir dizi döndürür.
- hstack: İki veya daha fazla diziyi yatay olarak (yan yana) yığar ve yeni bir dizi döndürür.

NumPy Dizileri Bölme (Splitting)

Birleştirmenin (concatenation) tam tersi, bölme (splitting) işlemidir ve bu işlem aşağıdaki fonksiyonlarla gerçekleştirilir:

- np.split()
- np.hsplit() (yatay bölme)
 - np.vsplit() (dikey bölme)

NumPy Dizileri Bölme (Splitting)

```
x = [1, 2, 3, 99, 99, 3, 2, 1]
x1, x2, x3 = np.split(x, [3, 5])
print(x1, x2, x3)

grid = np.arange(16).reshape((4, 4))
upper, lower = np.vsplit(grid, [2])
print(upper)
print(lower)

left, right = np.hsplit(grid, [2])
print(left)
print(right)
```

```
[1 2 3] [99 99] [3 2 1]

[[0 1 2 3]
 [4 5 6 7]]
[[ 8  9 10 11]
 [12 13 14 15]]

[[ 0  1]
 [ 4  5]
 [ 8  9]
 [12 13]]

[[ 2  3]
 [ 6  7]
 [10 11]
 [14 15]]
```

Figure 10: Dizileri Bölme

Dizileri Sıralama

Listelerde olduğu gibi, NumPy dizileri de `sort()` fonksiyonu kullanılarak yerinde (in-place) sıralanabilir.

```
In [67]: arr = np.random.rand(8)

In [68]: arr
Out[68]:
array([0.41857204, 0.52282439, 0.85841995, 0.97523084, 0.52313993,
       0.70206584, 0.37557734, 0.29295251])

In [69]: arr.sort()

In [70]: arr
Out[70]:
array([0.29295251, 0.37557734, 0.41857204, 0.52282439, 0.52313993,
       0.70206584, 0.85841995, 0.97523084])
```

Figure 11: Dizileri Sıralama

Benzersiz (Unique) ve Küme Mantığı (Set Logic)

Tek boyutlu diziler için, NumPy bazı temel küme işlemleri sağlar. En çok kullanılanı, bir dizideki sıralanmış benzersiz (unique) değerleri döndüren `unique()` fonksiyonudur.

```
In [72]: names = np.array(['Bob', 'Joe', 'Will', 'Bob', 'Will', 'Joe', 'Joe'])
```

```
In [73]: np.unique(names)
```

```
Out[73]: array(['Bob', 'Joe', 'Will'], dtype='<U4')
```

```
In [74]: ints = np.array([3,3,3,2,2,1,1,4,4])
```

```
In [75]: np.unique(ints)
```

```
Out[75]: array([1, 2, 3, 4])
```

Figure 12: Benzersiz (Unique)

Boolean dizi, True/False değerlerini içeren bir dizidir. NumPy'de, dizilerle eleman bazında ilişkisel ifadeler kullanılarak bir Boolean dizi oluşturulabilir.

```
In [87]: average_rainfall = np.array([55.0,71.3,10.5,0.0,8.6,30.2,18.8])  
  
In [88]: average_rainfall > 10  
Out[88]: array([ True,  True,  True, False, False,  True,  True])  
  
In [89]: (average_rainfall > 10) & (average_rainfall < 20)  
Out[89]: array([False, False,  True, False, False, False,  True])
```

Figure 13: Boolean Diziler)

Python, True değerlerini 1, False değerlerini ise 0 olarak kabul eder. Bir Boolean dizinin elemanlarını `sum()` fonksiyonu ile toplarsak, True değerlerinin sayısını elde ederiz.

```
In [106]: np.sum(average_rainfall > 10)
Out[106]: 5
```

Figure 14: Boolean Diziler)

Boolean İndeksleme ve Alt Diziler

Boolean dizileri kullanarak bir dizinin alt kümesini seçebiliriz. Örneğin, bir hafta boyunca yağış miktarının 10'dan fazla olduğu günleri bulmak istiyorsak, bunu mantıksal indeksleme (logical indexing) kullanarak yapabiliriz.

```
In [87]: average_rainfall = np.array([55.0,71.3,10.5,0.0,8.6,30.2,18.8])  
  
In [88]: average_rainfall > 10  
Out[88]: array([ True,  True,  True, False, False,  True,  True])  
  
In [90]: over_10 = average_rainfall > 10  
  
In [91]: average_rainfall[over_10]  
Out[91]: array([55. , 71.3, 10.5, 30.2, 18.8])
```

Figure 15: Boolean İndeksleme

where() fonksiyonu Dizi İndeksleri

Önceki örnekte, verilen koşulu(şartları) sağlayan değerleri/elemanları seçmek için mantıksal bir dizi kullandık. Eğer, verilen koşulu(şartları) sağlayan elemanların indekslerini bulmak istersek, bunu where() fonksiyonunu kullanarak yapabiliriz.

Aşağıdaki örnek, verilen koşulu sağlayan dizideki elemanların indekslerinin döndürüldüğünü göstermektedir. Yani, 0, 1, 2 vb. indisler, değeri 10'dan büyük olan elemanların konumlarını ifade eder.

```
In [96]: average_rainfall = np.array([55.0,71.3,10.5,0.0,8.6,30.2,18.8])  
  
In [97]: np.where(average_rainfall > 10)  
Out[97]: (array([0, 1, 2, 5, 6], dtype=int64),)
```

Figure 16: where() fonksiyonu

Alt Diziler ve Fonksiyonlar

```
import numpy as np
x = np.array([22,41,58,33,17,32])
#returns array of bool values for each element in x where condition is true
r1 = x > 40
print(r1)

#returns array containing elements in x where condition is true
r2 = x[x > 40]
print(r2)

#returns array containing indices of elements in x where condition is true
r3 = np.where(x > 40)
print(r3)
```

Figure 17: Alt Dizi ve Fonksiyonu

NumPy ile Dosyalardan Veri Okuma

NumPy, metin dosyalarını NumPy dizilerine okumamıza olanak tanıyan fonksiyonlar içerir.

`loadtxt(filename)` Belirtilen dosyadan (aksi belirtilmedikçe geçerli dizindeki) verileri bir NumPy dizisine okur. Varsayılan ayırıcı (delimiter) boşluktur. Varsayılan veri türü (dtype) float'tır. Dosyayı okurken ek parametreler ekleyebiliriz. Örneğin, eğer dosyanın ilk satırı başlık (header) içeriyorsa, ikinci satırdan itibaren okumak için `skiprows` parametresini kullanırız:

python Kodu kopyala `loadtxt('myfile.txt', skiprows=1)` Dosyadan dize (string) verilerini okumak istiyorsak, dtype parametresini kullanabiliriz:

python Kodu kopyala `loadtxt('myfile.txt', dtype='str')`

NumPy ile Dosyalara Veri Yazma

NumPy, NumPy dizilerini metin dosyalarına yazmamıza olanak tanıyan fonksiyonlar içerir.

`savetxt(filename, arrayName)` NumPy dizisindeki verileri bir metin dosyasına yazar. Varsayılan olarak, her satırdaki değerler boşlukla ayrılır ve satırlar yeni satır karakteriyle (`\n`) ayrılır. Dosyaya veri yazarken ek parametreler ekleyebiliriz. Tüm parametrelerin listesi ve açıklamaları için `savetxt` belgelendirmesine bakabilirsiniz.